

Product Requirements Document

Project: Offline UPI via SMS (OPAY) | Status: Architecture Design

1. Executive Summary

The **Offline UPI System (OPAY)** aims to facilitate seamless, secure financial transactions in zero-connectivity environments. By bridging a modern iOS application with a robust Java-based ledger via highly compressed, cryptographically secure SMS messages, the system provides banking accessibility without reliance on cellular data (3G/4G/5G) or Wi-Fi networks.

2. System Architecture

The solution is constructed upon a three-tier asynchronous architecture:

- **Client Tier (iOS):** A Swift-based application responsible for user interface, transaction queuing, hardware-backed cryptography, and SMS payload generation.
- **Gateway Tier (Android Wi-Fi Bridge):** A localized Android device running SMSGate, acting as a real-time HTTP-to-SMS webhook bridge on the local network.
- **Server Tier (Java Spring Boot):** The central banking simulator that handles cryptographic validation, transaction sequencing, and ledger management.

3. Functional Requirements

3.1. iOS Client (Swift)

- **Offline Promise Queue:** Transactions initiated without cellular data must be instantly written to an append-only local queue.
- **Auto-Dispatch:** The application must monitor cellular state and utilize `MFMessageComposeViewController` to dispatch queued transactions automatically upon signal acquisition.
- **Balance Inquiry:** Users must be able to request an encrypted balance check via SMS, updating the local UI upon receiving the server's response.

3.2. Android Gateway

- **Webhook Integration:** Must listen for local HTTP POST requests from the Server Tier and forward the payload as an SMS.

- **SMS Catch & Forward:** Must intercept incoming SMS matching the designated protocol header and forward the JSON payload to the Java Server's REST endpoint.

3.3. Java Spring Boot Server

- **Decryption & Validation:** Must parse the Base85 payload, enforce time-window constraints, and cryptographically verify the signature against the sender's public key.
- **Ledger Management:** Must atomize transactions (check balance, deduct sender, credit receiver, record Txn ID) to prevent double-spending or rollback failures.

4. Cryptography & Security Constraints

The 160-Character SMS Bottleneck

Due to standard GSM limitations, all cryptographic material (Sender, Receiver, Amount, Timestamp, Signature) must be aggressively optimized. Base64 introduces a 33% overhead; therefore, **Base85 (Ascii85)** encoding is mandated to maintain the payload within a single SMS frame.

- **Secure Enclave Binding:** Device identity must not rely on spoofable firmware IDs. A unique ECDSA key pair must be generated within the iOS Secure Enclave during onboarding. The private key never leaves the hardware.
- **Time-Scoped Encryption:** Payload encryption (ECDH) must utilize time-rotating keys synchronized with the server to guarantee forward secrecy.
- **Tamper-Proof Audit Trail:** The local offline queue must utilize a cryptographic Hash-Chain. Each transaction payload must include the SHA-256 hash of the preceding transaction, ensuring chronological integrity.
- **Carrier Spam Evasion:** All SMS payloads must be prepended with a clear text header (e.g., `OPAY-TXN:`) to prevent aggressive telecom carrier filtering.

5. Data Storage & Algorithmic Specifications

5.1. Database Technologies

Tier	Database Engine	Primary Use Case
iOS Client	Core Data (SQLite)	User metadata, Offline Promise Queue, Local History
Android Gateway	Room (SQLite)	Routing logs, HTTP/SMS state tracking

Tier	Database Engine	Primary Use Case
Java Server	PostgreSQL / H2	Core banking ledger, Processed Txn IDs, Public Keys

5.2. Core Algorithms

- **Replay Attack Prevention:** Server utilizes $O(1)$ Hash Table Lookups (e.g., Redis or in-memory HashSet) to instantly identify and reject duplicated Transaction IDs.
- **Ledger Query Optimization:** PostgreSQL schema must index the phone number and transaction IDs, ensuring $O(\log n)$ B-Tree searching during high-volume message bursts.
- **Queue Processing:** iOS client utilizes a Priority Queue (Min-Heap, $O(n \log n)$) sorted by Timestamp to ensure delayed offline transactions are submitted to the SMS interface in the exact chronological order of creation.

6. Payload Serialization Schema

To fit within 160 characters, the binary packet structure prior to Base85 encoding is defined as:

Field	Byte Size	Description
Version / Flag	1 Byte	Protocol version and Txn Type (Balance vs Transfer)
Recipient ID	4 Bytes	Truncated/Hashed phone number mapping
Amount	2-4 Bytes	Integer value of transfer
Timestamp	4 Bytes	Unix epoch for Replay protection
Txn ID + Hash Link	8 Bytes	Unique sequence + Hash of previous Txn
ECDSA Signature	~64 Bytes	Secure Enclave signature of the above bytes

Note: The resulting binary (approx. 85 bytes) is encrypted, encoded in Base85, appended to the text header, resulting in a ~120-character final SMS.